EXPT NO: 9 DATE: 2/11/2026	TEXT CLASSIFICATION AND TEXT GENERATION USING TRANSFORMERS

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Transformer model in Natural Language Processing (NLP)?

A Transformer is a deep learning architecture used in NLP that uses **self-attention** to understand relationships between words in a sentence. It processes multiple tokens in parallel and captures contextual information efficiently.

Key components:

- **Encoder:** Processes input text and creates contextual representations.
- **Decoder:** Generates output text.
- **Self-attention:** Identifies relationships between words.
- **Positional encoding:** Provides information about token positions.
  Applications: Machine translation, text classification, text summarization, and text generation.

2. What is BERT, and how is it used for text classification?

BERT stands for **Bidirectional Encoder Representations from Transformers**. It is a pretrained language model developed by Google that understands words by considering both left and right context.

Working:

1. Input text is converted into tokens.
2. BERT processes the tokens and captures contextual information.
3. A classification layer predicts the category.
4. The output is assigned a label.
   Example: The sentence "This movie is excellent" is classified as **Positive Sentiment**.
   Applications: Sentiment analysis, spam detection, emotion detection, and news classification.

3. What is fine-tuning in the context of pretrained language models?

Fine-tuning is the process of training a pretrained language model further on a task-specific dataset to improve its performance on a particular task.

Steps:

1. Select a pretrained model such as BERT.
2. Prepare a task-specific dataset.
3. Train the model using the dataset.
4. Evaluate its performance.
   Example: Fine-tuning BERT with labeled emails enables it to classify emails as spam or not spam.
   Advantages: Saves training time, requires less data than training from scratch in many cases, and improves task-specific performance.

4. What is the difference between text classification and text generation?

Feature	Text Classification	Text Generation
Definition	Assigns predefined categories to text.	Generates new text based on input or context.
Purpose	Identifies the class or category.	Produces meaningful sentences or paragraphs.
Input	Text or document.	Prompt, text, or context.
Output	A class label or category.	A sequence of words or sentences.
Models	BERT, DistilBERT.	GPT, T5.
Example	Classifying a review as positive or negative.	Generating a review or story.

Applications	Spam detection, sentiment analysis.	Story writing, chatbots, text summarization.

5. What is a sequence-to-sequence model, and how is it used in text generation?

A **Sequence-to-Sequence (Seq2Seq) model** is a deep learning architecture that converts an input sequence into an output sequence. The input and output may have different lengths.

Main components:

- **Encoder:** Processes the input sequence and creates contextual representations.
- **Decoder:** Generates the output sequence one token at a time using the encoded information and previously generated tokens.

Working:

1. The encoder processes the input text.
2. The decoder uses the encoded information to generate output tokens.
3. Tokens are generated sequentially until the output is complete.

Example: English input "How are you?" is translated into Tamil as "நீங்கள் எப்படி இருக்கிறீர்கள்?"

Applications: Machine translation, text summarization, chatbots, and dialogue generation.

Examples of models: Traditional RNN/LSTM-based Seq2Seq models and Transformer-based models such as T5.

AIM

To implement text classification by fine-tuning a pretrained BERT model and develop a simple sequence-to-sequence text generation application using Transformer models.

SOFTWARE REQUIREMENTS

- Python
- Google Colab
- Hugging Face Transformers
- Hugging Face Datasets
- PyTorch
- NumPy
- Pandas
- Matplotlib

OBJECTIVES

Students will be able to:

- Understand the architecture and working of Transformer models in NLP.
- Understand the concepts of text classification and text generation.
- Load and preprocess text datasets for NLP applications.
- Fine-tune a pretrained BERT model for text classification.
- Evaluate the classification model using appropriate performance metrics.
- Implement a sequence-to-sequence application using a pretrained Transformer model.
- Generate text using the model's text generation capabilities.
- Analyze and compare the outputs produced by the Transformer models.

PROCEDURE

PART A: DATASET PREPARATION AND TEXT PREPROCESSING

Perform the following tasks:

- Load a text dataset such as the IMDb movie review dataset.
- Explore the dataset and examine sample text entries and their labels.
- Check the number of samples and the distribution of class labels.
- Separate the dataset into training and testing sets or use the predefined splits.

- Load a pretrained tokenizer corresponding to the BERT model.
- Tokenize the text using the tokenizer.
- Apply padding and truncation to maintain consistent input lengths.
- Convert the tokenized data into the required format for model training.
- Prepare the dataset for batch-wise processing.

PART B: FINE-TUNING BERT FOR TEXT CLASSIFICATION

Perform the following tasks:

- Load a pretrained BERT model for sequence classification.
- Configure the model for the required number of classes.
- Prepare the tokenized training and testing datasets.
- Define the training parameters, including learning rate, batch size, and number of epochs.
- Fine-tune the BERT model using the training dataset.
- Evaluate the trained model using the testing dataset.
- Calculate appropriate performance metrics, such as accuracy, precision, recall, and F1-score.
- Test the model using new text inputs.
- Display the predicted class labels and corresponding evaluation results.
- Save the fine-tuned model and tokenizer.

PART C: IMPLEMENTING A SEQUENCE-TO-SEQUENCE APPLICATION USING TRANSFORMERS

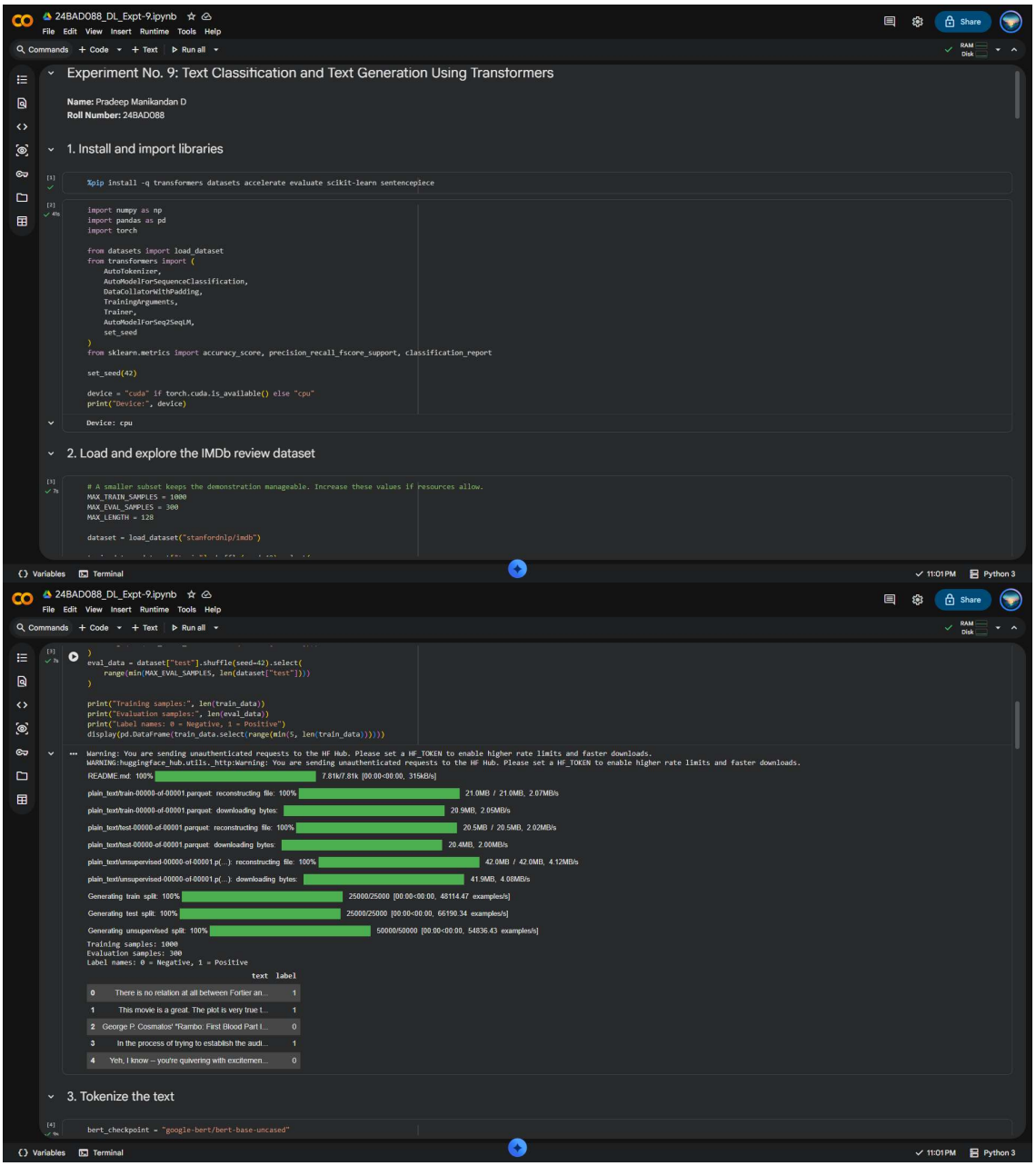
Perform the following tasks:

- Load a pretrained sequence-to-sequence Transformer model, such as T5-small.
- Load the corresponding tokenizer.
- Prepare a sample text input for the selected text generation task.
- Preprocess the input according to the model's requirements.
- Generate output text using the pretrained model.
- Apply appropriate generation parameters, such as maximum output length and beam search, where applicable.
- Display the input text and the generated output.
- Experiment with different input texts and generation settings.
- Observe how changes in the input and generation parameters affect the output.
- Save the generated outputs for further analysis.

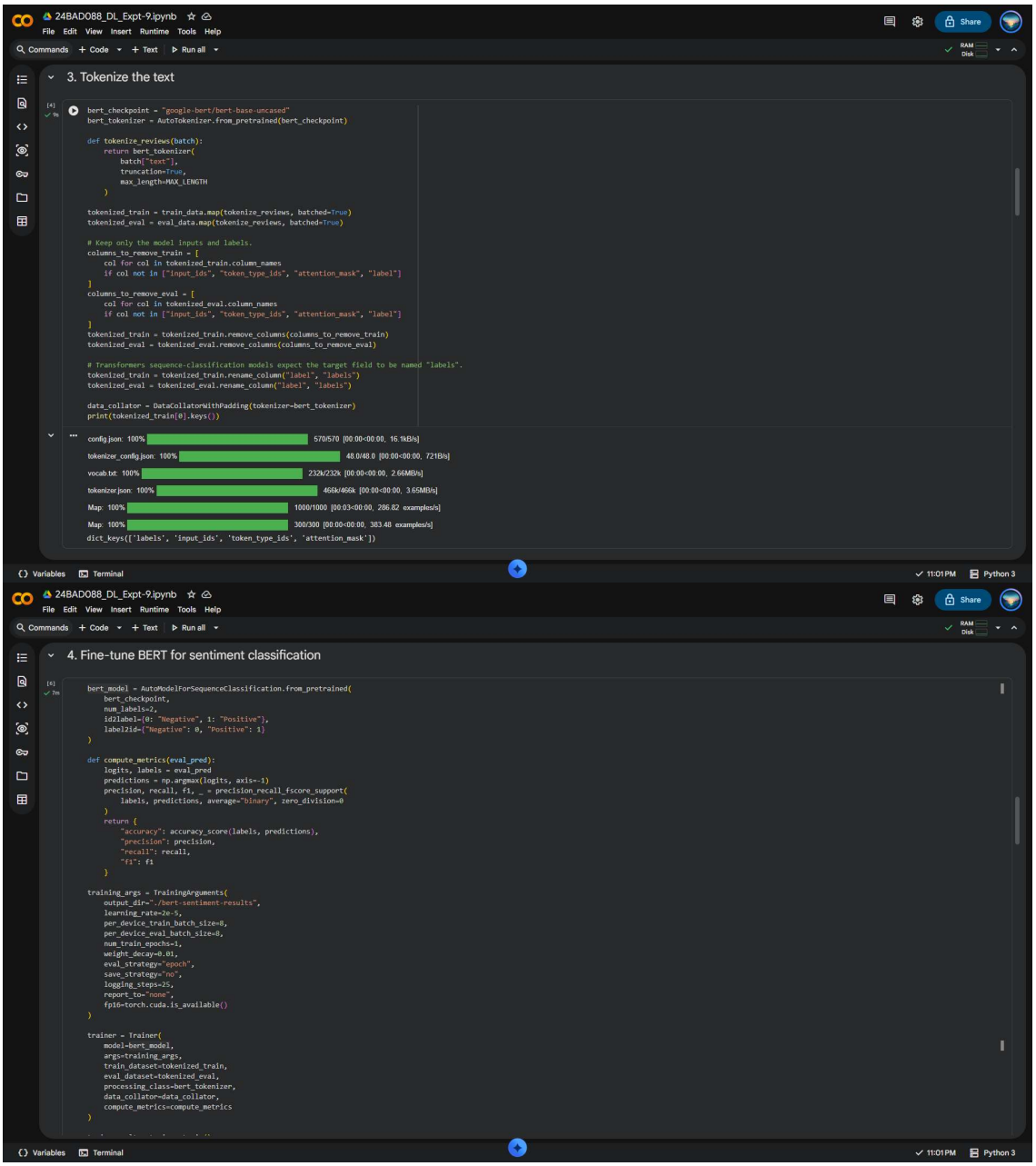
PART D: MODEL TESTING AND OUTPUT ANALYSIS

Perform the following tasks:

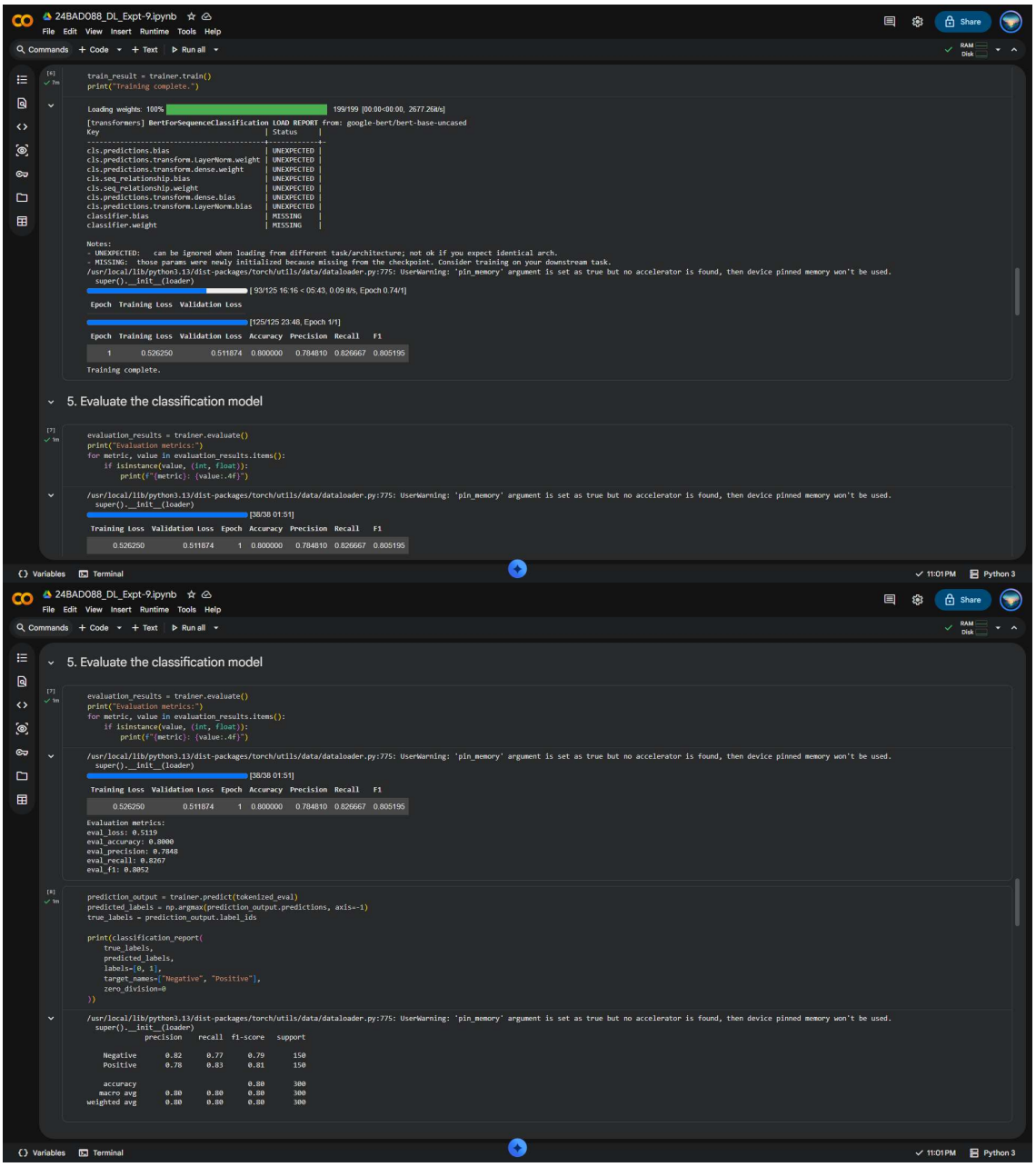
- Test the fine-tuned BERT model using sample positive and negative text inputs.
- Record the predicted class labels for the test inputs.
- Analyze the classification results using the selected evaluation metrics.
- Test the sequence-to-sequence model using different input texts.
- Compare the generated outputs for different inputs and generation settings.
- Examine the relevance, readability, and coherence of the generated text.
- Identify possible errors or limitations in the classification and generation outputs.
- Compare the applications of BERT-based classification and sequence-to-sequence text generation.
- Summarize the observations obtained from both models.

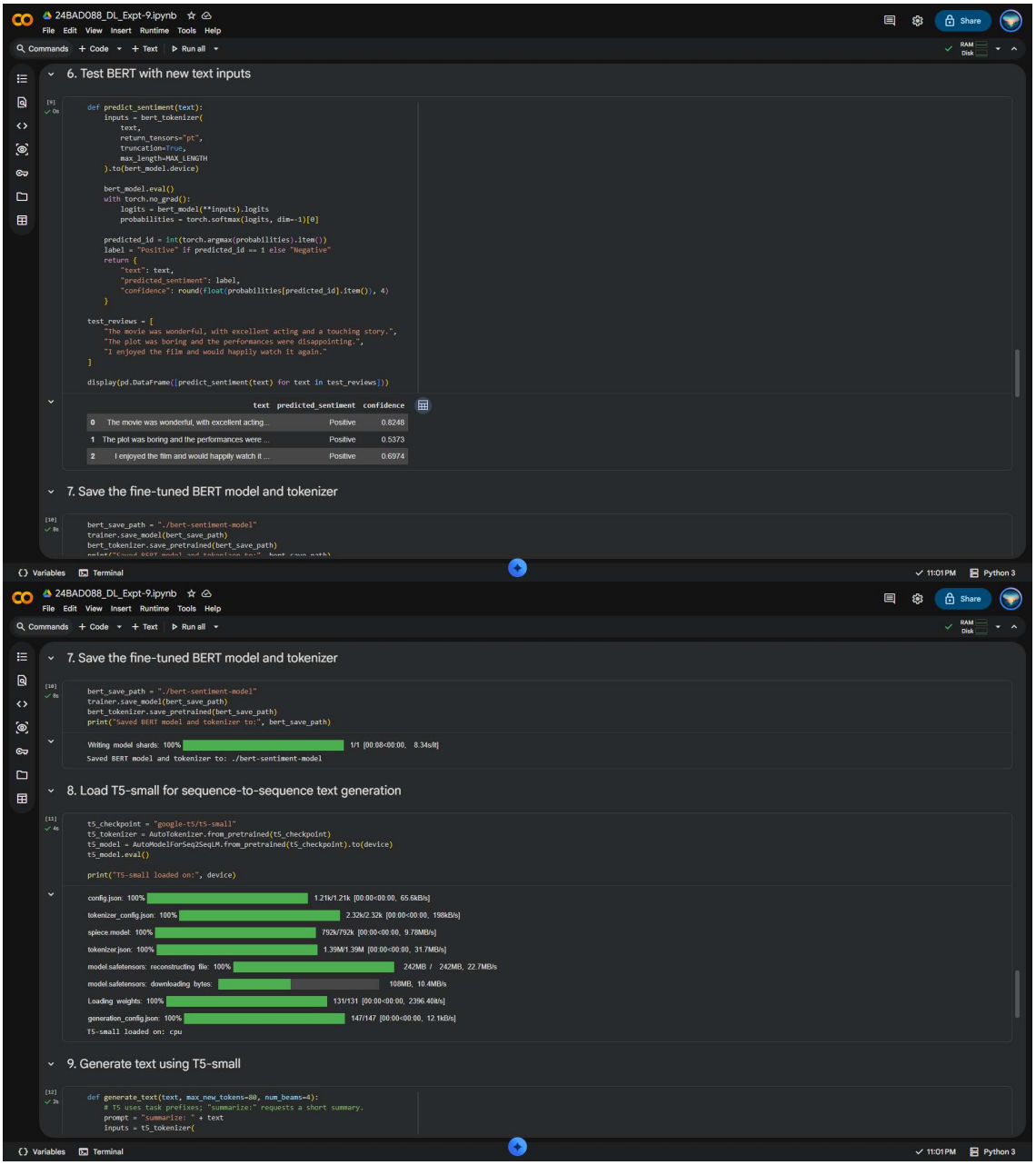

24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Experiment No. 9: Text Classification and Text Generation Using Transformers
Name: Pradeep Manikandan D
Roll Number: 24BAD088
1. Install and import libraries
%pip install -q transformers datasets accelerate evaluate scikit-learn sentencepiece
import numpy as np
import pandas as pd
import torch
from datasets import load_dataset
from transformers import (
AutoTokenizer,
AutoModelForSequenceClassification,
DataCollatorWithPadding,
TrainingArguments,
Trainer,
AutoModelForSeq2SeqLM,
set_seed
)
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, classification_report
set_seed(42)
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", device)
Device: cpu
2. Load and explore the IMDb review dataset
A smaller subset keeps the demonstration manageable. Increase these values if resources allow.
MAX_TRAIN_SAMPLES = 1000
MAX_EVAL_SAMPLES = 300
MAX_LENGTH = 128
dataset = load_dataset("stanfordnlp/imdb")
Variables Terminal
11:01 PM Python 3

24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
)
eval_data = dataset["test"].shuffle(seed=42).select(
range(min(MAX_EVAL_SAMPLES, len(dataset["test"])))
)
print("Training samples:", len(train_data))
print("Evaluation samples:", len(eval_data))
print("Label names: 0 = Negative, 1 = Positive")
display(pd.DataFrame(train_data.select(range(min(5, len(train_data))))))
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md: 100% 7.81k/7.81k [00:00<00:00, 315kB/s]
plain_text/train-00000-of-00001.parquet: reconstructing file: 100% 21.0MB / 21.0MB, 2.07MB/s
plain_text/train-00000-of-00001.parquet: downloading bytes: 20.9MB, 2.05MB/s
plain_text/test-00000-of-00001.parquet: reconstructing file: 100% 20.5MB / 20.5MB, 2.02MB/s
plain_text/test-00000-of-00001.parquet: downloading bytes: 20.4MB, 2.00MB/s
plain_text/unsupervised-00000-of-00001.p(…): reconstructing file: 100% 42.0MB / 42.0MB, 4.12MB/s
plain_text/unsupervised-00000-of-00001.p(…): downloading bytes: 41.9MB, 4.08MB/s
Generating train split: 100% 25000/25000 [00:00<00:00, 48114.47 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 66190.34 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 54836.43 examples/s]
Training samples: 1000
Evaluation samples: 300
Label names: 0 = Negative, 1 = Positive
text label
0 There is no relation at all between Fortier an... 1
1 This movie is a great. The plot is very true t... 1
2 George P. Cosmatos' "Rambo: First Blood Part I... 0
3 In the process of trying to establish the audi... 1
4 Yeh, I know -- you're quivering with excitemen... 0
3. Tokenize the text
bert_checkpoint = "google-bert/bert-base-uncased"
Variables Terminal
11:01 PM Python 3

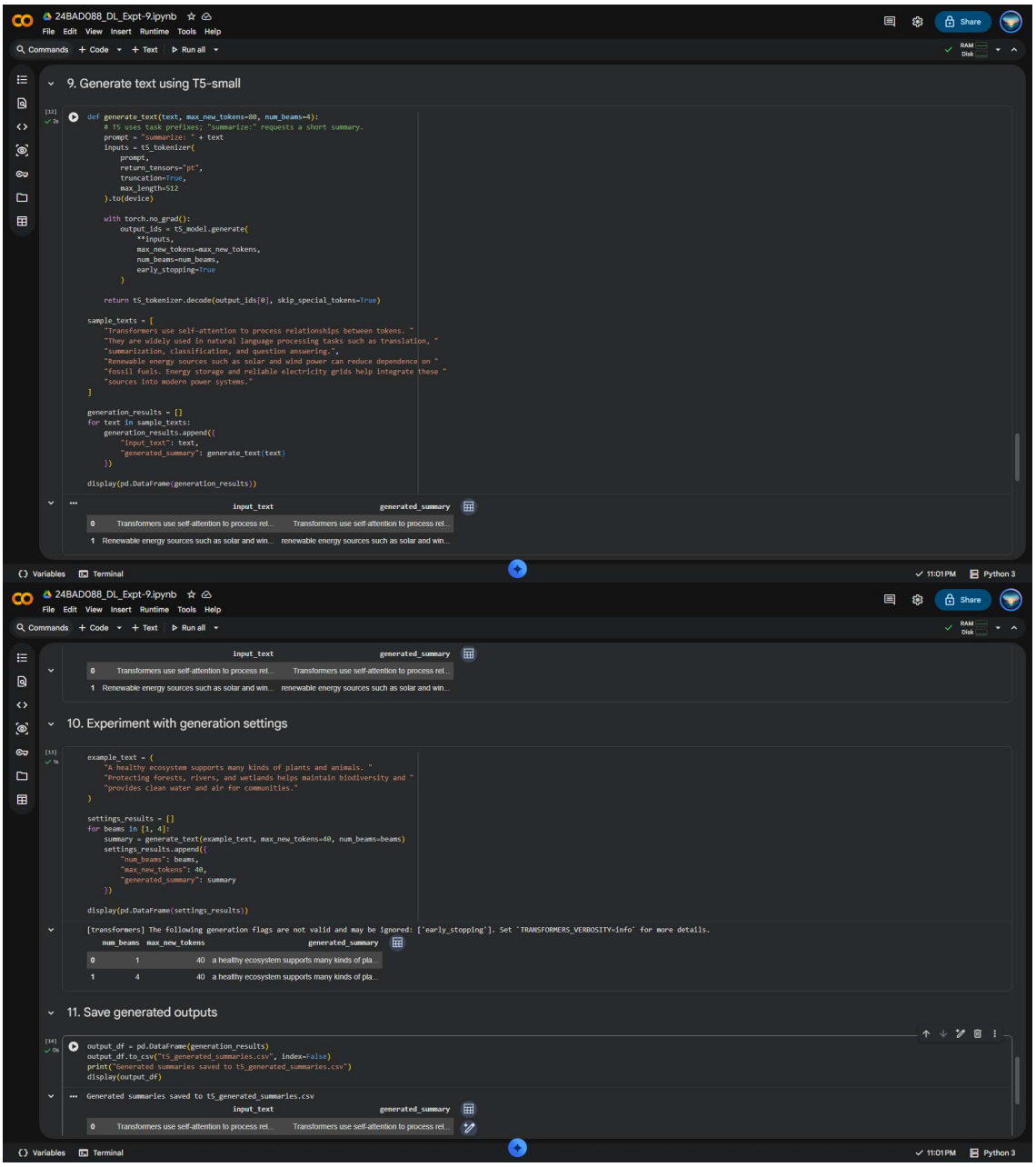

24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
3. Tokenize the text
bert_checkpoint = "google-bert/bert-base-uncased"
bert_tokenizer = AutoTokenizer.from_pretrained(bert_checkpoint)
def tokenize_reviews(batch):
return bert_tokenizer(
batch["text"],
truncation=True,
max_length=MAX_LENGTH
)
tokenized_train = train_data.map(tokenize_reviews, batched=True)
tokenized_eval = eval_data.map(tokenize_reviews, batched=True)
Keep only the model inputs and labels.
columns_to_remove_train = [
col for col in tokenized_train.column_names
if col not in ["input_ids", "token_type_ids", "attention_mask", "label"]
]
columns_to_remove_eval = [
col for col in tokenized_eval.column_names
if col not in ["input_ids", "token_type_ids", "attention_mask", "label"]
]
tokenized_train = tokenized_train.remove_columns(columns_to_remove_train)
tokenized_eval = tokenized_eval.remove_columns(columns_to_remove_eval)
Transformers sequence-classification models expect the target field to be named "labels".
tokenized_train = tokenized_train.rename_column("label", "labels")
tokenized_eval = tokenized_eval.rename_column("label", "labels")
data_collator = DataCollatorWithPadding(tokenizer=bert_tokenizer)
print(tokenized_train[0].keys())
config.json: 100% 570/570 [00:00<00:00, 16.1kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 721B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.66MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 3.65MB/s]
Map: 100% 1000/1000 [00:03<00:00, 286.82 examples/s]
Map: 100% 300/300 [00:00<00:00, 383.48 examples/s]
dict_keys(['labels', 'input_ids', 'token_type_ids', 'attention_mask'])
Variables Terminal
11:01 PM Python 3

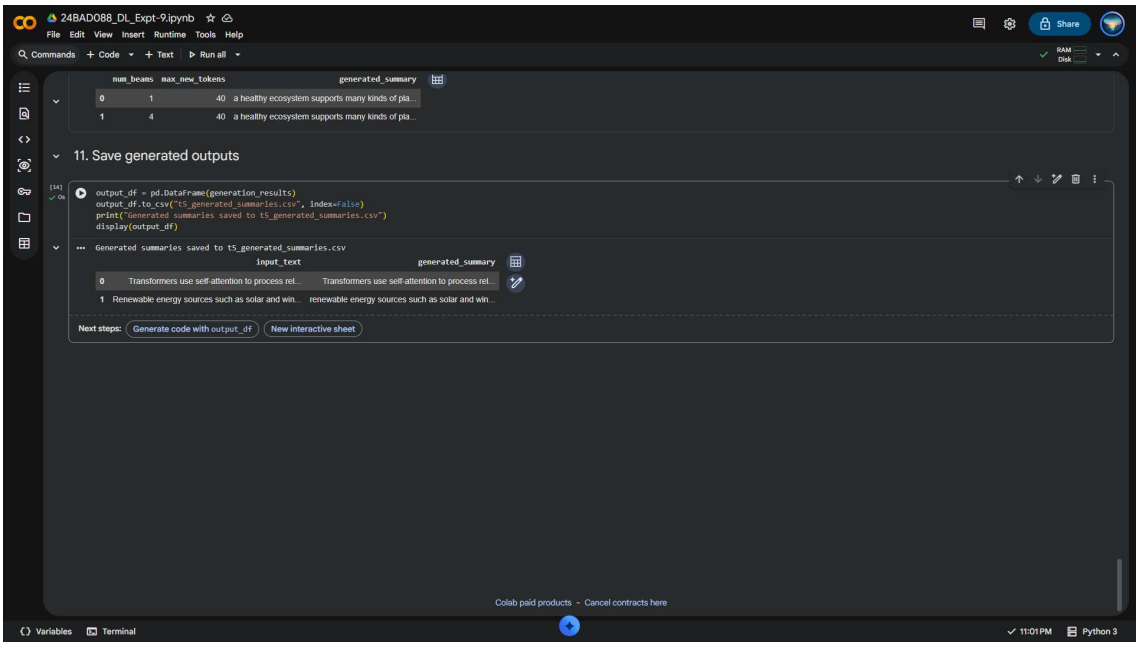
24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
4. Fine-tune BERT for sentiment classification
bert_model = AutoModelForSequenceClassification.from_pretrained(
bert_checkpoint,
num_labels=2,
id2label={0: "Negative", 1: "Positive"},
label2id={"Negative": 0, "Positive": 1}
)
def compute_metrics(eval_pred):
logits, labels = eval_pred
predictions = np.argmax(logits, axis=-1)
precision, recall, f1, _ = precision_recall_fscore_support(
labels, predictions, average="binary", zero_division=0
)
return {
"accuracy": accuracy_score(labels, predictions),
"precision": precision,
"recall": recall,
"f1": f1
}
training_args = TrainingArguments(
output_dir="./bert-sentiment-results",
learning_rate=2e-5,
per_device_train_batch_size=8,
per_device_eval_batch_size=8,
num_train_epochs=1,
weight_decay=0.01,
eval_strategy="epoch",
save_strategy="no",
logging_steps=25,
report_to="none",
fp16=torch.cuda.is_available()
)
trainer = Trainer(
model=bert_model,
args=training_args,
train_dataset=tokenized_train,
eval_dataset=tokenized_eval,
processing_class=bert_tokenizer,
data_collator=data_collator,
compute_metrics=compute_metrics
)
Variables Terminal
11:01 PM Python 3


24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
train_result = trainer.train()
print("Training complete.")
Loading weights: 100% 199/199 [00:00<00:00, 2677.26it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: google-bert/bert-base-uncased
Key | Status |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |
Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
[93/125 16:16 < 05:43, 0.09 it/s, Epoch 0.74/1]
Epoch Training Loss Validation Loss
[125/125 23:48, Epoch 1/1]
Epoch Training Loss Validation Loss Accuracy Precision Recall F1
1 0.526250 0.511874 0.800000 0.784810 0.826667 0.805195
Training complete.
5. Evaluate the classification model
evaluation_results = trainer.evaluate()
print("Evaluation metrics:")
for metric, value in evaluation_results.items():
if isinstance(value, (int, float)):
print(f"{metric}: {value:.4f}")
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
[38/38 01:51]
Training Loss Validation Loss Epoch Accuracy Precision Recall F1
0.526250 0.511874 1 0.800000 0.784810 0.826667 0.805195
Variables Terminal
11:01 PM Python 3
24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
5. Evaluate the classification model
evaluation_results = trainer.evaluate()
print("Evaluation metrics:")
for metric, value in evaluation_results.items():
if isinstance(value, (int, float)):
print(f"{metric}: {value:.4f}")
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
[38/38 01:51]
Training Loss Validation Loss Epoch Accuracy Precision Recall F1
0.526250 0.511874 1 0.800000 0.784810 0.826667 0.805195
Evaluation metrics:
eval_loss: 0.5119
eval_accuracy: 0.8000
eval_precision: 0.7848
eval_recall: 0.8267
eval_f1: 0.8052
prediction_output = trainer.predict(tokenized_eval)
predicted_labels = np.argmax(prediction_output.predictions, axis=-1)
true_labels = prediction_output.label_ids
print(classification_report(
true_labels,
predicted_labels,
labels=[0, 1],
target_names=["Negative", "Positive"],
zero_division=0
))
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
precision recall f1-score support
Negative 0.82 0.77 0.79 150
Positive 0.78 0.83 0.81 150
accuracy 0.80 300
macro avg 0.80 0.80 0.80 300
weighted avg 0.80 0.80 0.80 300
Variables Terminal
11:01 PM Python 3


24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
6. Test BERT with new text inputs
def predict_sentiment(text):
inputs = bert_tokenizer(
text,
return_tensors="pt",
truncation=True,
max_length=MAX_LENGTH
).to(bert_model.device)
bert_model.eval()
with torch.no_grad():
logits = bert_model(**inputs).logits
probabilities = torch.softmax(logits, dim=-1)[0]
predicted_id = int(torch.argmax(probabilities).item())
label = "Positive" if predicted_id == 1 else "Negative"
return {
"text": text,
"predicted_sentiment": label,
"confidence": round(float(probabilities[predicted_id].item()), 4)
}
test_reviews = [
"The movie was wonderful, with excellent acting and a touching story.",
"The plot was boring and the performances were disappointing.",
"I enjoyed the film and would happily watch it again."
]
display(pd.DataFrame([predict_sentiment(text) for text in test_reviews]))
text predicted_sentiment confidence
0 The movie was wonderful, with excellent acting... Positive 0.8248
1 The plot was boring and the performances were ... Positive 0.5373
2 I enjoyed the film and would happily watch it ... Positive 0.6974
7. Save the fine-tuned BERT model and tokenizer
bert_save_path = "./bert-sentiment-model"
trainer.save_model(bert_save_path)
bert_tokenizer.save_pretrained(bert_save_path)
Variables Terminal
11:01 PM Python 3

24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
7. Save the fine-tuned BERT model and tokenizer
bert_save_path = "./bert-sentiment-model"
trainer.save_model(bert_save_path)
bert_tokenizer.save_pretrained(bert_save_path)
print("Saved BERT model and tokenizer to:", bert_save_path)
Writing model shards: 100% 1/1 [00:08<00:00, 8.34s/it]
Saved BERT model and tokenizer to: ./bert-sentiment-model
8. Load T5-small for sequence-to-sequence text generation
t5_checkpoint = "google-t5/t5-small"
t5_tokenizer = AutoTokenizer.from_pretrained(t5_checkpoint)
t5_model = AutoModelForSeq2SeqLM.from_pretrained(t5_checkpoint).to(device)
t5_model.eval()
print("T5-small loaded on:", device)
config.json: 100% 1.21k/1.21k [00:00<00:00, 65.6kB/s]
tokenizer_config.json: 100% 2.32k/2.32k [00:00<00:00, 198kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 9.78MB/s]
tokenizer.json: 100% 1.39M/1.39M [00:00<00:00, 31.7MB/s]
model.safetensors: reconstructing file: 100% 242MB / 242MB, 22.7MB/s
model.safetensors: downloading bytes: 108MB, 10.4MB/s
Loading weights: 100% 131/131 [00:00<00:00, 2396.40it/s]
generation_config.json: 100% 147/147 [00:00<00:00, 12.1kB/s]
T5-small loaded on: cpu
9. Generate text using T5-small
def generate_text(text, max_new_tokens=80, num_beams=4):
T5 uses task prefixes; "summarize:" requests a short summary.
prompt = "summarize: " + text
inputs = t5_tokenizer(
Variables Terminal
11:01 PM Python 3


24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
9. Generate text using T5-small
def generate_text(text, max_new_tokens=80, num_beams=4):
T5 uses task prefixes; "summarize:" requests a short summary.
prompt = "summarize: " + text
inputs = t5_tokenizer(
prompt,
return_tensors="pt",
truncation=True,
max_length=512
).to(device)
with torch.no_grad():
output_ids = t5_model.generate(
**inputs,
max_new_tokens=max_new_tokens,
num_beams=num_beams,
early_stopping=True
)
return t5_tokenizer.decode(output_ids[0], skip_special_tokens=True)
sample_texts = [
"Transformers use self-attention to process relationships between tokens. "
"They are widely used in natural language processing tasks such as translation, "
"summarization, classification, and question answering.",
"Renewable energy sources such as solar and wind power can reduce dependence on "
"fossil fuels. Energy storage and reliable electricity grids help integrate these "
"sources into modern power systems."
]
generation_results = []
for text in sample_texts:
generation_results.append({
"input_text": text,
"generated_summary": generate_text(text)
})
display(pd.DataFrame(generation_results))
input_text generated_summary
0 Transformers use self-attention to process rel... Transformers use self-attention to process rel...
1 Renewable energy sources such as solar and win... renewable energy sources such as solar and win...
Variables Terminal
11:01 PM Python 3
24BAD088_DL_Expt-9.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
input_text generated_summary
0 Transformers use self-attention to process rel... Transformers use self-attention to process rel...
1 Renewable energy sources such as solar and win... renewable energy sources such as solar and win...
10. Experiment with generation settings
example_text = (
"A healthy ecosystem supports many kinds of plants and animals. "
"Protecting forests, rivers, and wetlands helps maintain biodiversity and "
"provides clean water and air for communities."
)
settings_results = []
for beams in [1, 4]:
summary = generate_text(example_text, max_new_tokens=40, num_beams=beams)
settings_results.append({
"num_beams": beams,
"max_new_tokens": 40,
"generated_summary": summary
})
display(pd.DataFrame(settings_results))
[transformers] The following generation flags are not valid and may be ignored: ['early_stopping']. Set `TRANSFORMERS_VERBOSITY=info` for more details.
num_beams max_new_tokens generated_summary
0 1 40 a healthy ecosystem supports many kinds of pla...
1 4 40 a healthy ecosystem supports many kinds of pla...
11. Save generated outputs
output_df = pd.DataFrame(generation_results)
output_df.to_csv("t5_generated_summaries.csv", index=False)
print("Generated summaries saved to t5_generated_summaries.csv")
display(output_df)
Generated summaries saved to t5_generated_summaries.csv
input_text generated_summary
0 Transformers use self-attention to process rel... Transformers use self-attention to process rel...
Variables Terminal
11:01 PM Python 3



EXPECTED OUTCOME

Students will understand the architecture and applications of Transformer models in Natural Language Processing. They will learn to fine-tune a pretrained BERT model for text classification, preprocess textual data, and evaluate classification performance using appropriate metrics. Students will also develop a simple sequence-to-sequence text generation application using a pretrained Transformer model and analyze the quality and relevance of the generated outputs.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is the role of BERT in text classification?

BERT (Bidirectional Encoder Representations from Transformers) understands the contextual meaning of words by considering both left and right context. In text classification, BERT processes the input text and uses a classification layer to predict categories such as positive sentiment, negative sentiment, or spam.

2. Why is tokenization required before providing text to a Transformer model?

Tokenization converts input text into smaller units called tokens, such as words or subwords. These tokens are mapped to numerical IDs that the Transformer can process. Tokenization enables the model to represent text numerically and understand linguistic patterns.

3. What is the purpose of fine-tuning a pretrained BERT model?

Fine-tuning adapts a pretrained BERT model to a specific task using task-specific training data. It updates the model's parameters to improve performance on tasks such as sentiment analysis, spam detection, and text classification without training the model from scratch.

4. Mention any two evaluation metrics used for text classification.

Two evaluation metrics are **accuracy and precision**. Accuracy measures the proportion of correctly classified instances among all instances. Precision measures the proportion of correctly predicted positive instances among all instances predicted as positive. Both metrics help evaluate classification performance.

5. What is the role of the encoder and decoder in a sequence-to-sequence Transformer?
The **encoder** processes the input sequence and generates contextual representations using self-attention. The **decoder** uses these representations and previously generated tokens to produce the output sequence one token at a time. Together, they support tasks such as machine translation and text summarization.

6. What is the difference between text classification and text generation?

Text Classification	Text Generation
Assigns predefined labels to input text.	Produces new text based on input or context.
Example: Sentiment analysis.	Example: Story writing.
Models: BERT, DistilBERT.	Model: GPT.

7. How does the maximum output length affect text generation?
Maximum output length determines the number of tokens a model can generate. A shorter limit may truncate the response, while a longer limit allows more detailed output. However, longer outputs require additional computational resources and may contain unnecessary information.

8. Mention one real-world application of Transformer-based text classification and one application of text generation.
Text classification: Email spam detection uses Transformer models such as BERT to classify emails as spam or legitimate. **Text generation:** AI chatbots use models such as GPT to generate human-like responses to user questions, enabling automated customer support and conversational assistance.

SUGGESTIONS FOR DATASETS & MODELS
Datasets:

- IMDb Movie Reviews – https://huggingface.co/datasets/stanfordnlp/imdb
- GLUE Benchmark – https://huggingface.co/datasets/nyu-mll/glue
- SST-2 Sentiment Classification – https://huggingface.co/datasets/nyu-mll/glue

Pretrained Models:

- BERT Base Uncased – https://huggingface.co/google-bert/bert-base-uncased
- T5-small – https://huggingface.co/google-t5/t5-small

GitHub: https://github.com/Pradeep-Manikandan-1-D/24BAD088_DEEP_LEARNING_LAB

ASSESSMENT

Description	Max Marks	Marks Awarded	
Pre Lab Exercise	**5**		
In Lab Exercise	**10**		
Post Lab Exercise	**5**		
Viva	**10**		
Total	**30**		
Faculty Signature			